

# QuickSort Performance Analysis Project (Median-of-Three)

## 1. Project Overview

This project implements the QuickSort algorithm with a **Median-of-Three partitioning strategy** and includes a complete automated testing framework to measure its average-case performance.

The project automatically generates 75 input files across three sizes (10, 100, 1000), sorts them, records the execution time for each run, and calculates the average time for each size group.

## 2. Project Files

| File Name                                           | Description                                                                                                                                                                     |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Austin_Jerard_QuickSort.cc</code>             | The core C++ implementation of the QuickSort algorithm (using Median-of-Three partitioning). It reads input, sorts data, and logs the execution time to a single raw data file. |
| <code>InputFileGenerator.cc</code>                  | C++ utility to create 75 random input files (25 files for each size: 10, 100, 1000).                                                                                            |
| <code>TimeCalculator.cc</code>                      | C++ utility that reads the raw execution data, calculates the average running time for each size group, and generates the final report.                                         |
| <code>Makefile</code>                               | The central build script used to compile all C++ source files and automate the entire testing and analysis workflow.                                                            |
| <code>ChartPlot.py</code>                           | Python script using <code>matplotlib</code> to read the final average times and generate a performance plot.                                                                    |
| <code>Austin_Jerard_executionTime.txt</code>        | <b>Generated file</b> containing raw time data for all 75 runs (size and time per line).                                                                                        |
| <code>Austin_Jerard_averageExecutionTime.txt</code> | <b>Generated file</b> containing the final calculated average execution times for sizes 10, 100, and 1000.                                                                      |

## 3. Build and Execution

The entire process is managed via the `Makefile`. You will use three main targets: `all`, `test`, and `average`.

### 3.1. Prerequisite

Before running the Python script, ensure the `matplotlib` library is installed:

```
pip install matplotlib
```

### 3.2. Build All Executables

This compiles all three C++ source files ( `QuickSort` , `GenFile` , `AvgTime` ).

```
make all
```

### 3.3. Run Automated Tests

This command runs the full 75-run test suite. It generates all input files and executes `Austin_Jerard_QuickSort` 75 times, logging the size and time for each run into the raw data file.

```
make test
```

### 3.4. Calculate Averages and Generate Report

This is the primary command for analysis. It first ensures tests are complete ( `test` ), then compiles and runs `AvgTime` to calculate the averages and produce the final report.

```
make average
```

### 3.5. Generate Performance Plot (Visualization)

After successfully running `make average` , the final data is ready for plotting.

```
python3 ChartPlot.py
```

## 4. Cleaning the Project

This removes all generated files (executables and all `.txt` data files) to prepare for a clean rebuild or submission.

```
make clean
```